

第 29 讲实验设计文档

PyTorch Autograd：从手工 Backprop 到自动微分

一、实验目的

本实验不是训练一个高性能模型，而是验证 PyTorch Autograd 是否与我们已经掌握的手工 Backpropagation 一致，并观察梯度记录、累加、切断和 Shape 规则。

二、研究问题 (RQ)

1. RQ1: requires_grad 开启后，计算图中 leaf / non-leaf / grad_fn 呈现什么关系？
2. RQ2: Autograd 对简单函数与单神经元计算出的梯度，能否与解析/手工梯度一致？
3. RQ3: 为什么连续 backward 会导致梯度累加？
4. RQ4: 非标量输出为什么需要额外定义上游梯度或聚合方式？
5. RQ5: nn.Linear + MSE 的 Autograd 梯度能否与矩阵公式严格对齐？
6. RQ6: no_grad 与 detach 是否真的会切断梯度记录？

三、实验假设

- H1: 对 $L = x^2, x = 3$, Autograd 应得到 $dL/dx = 6$ 。
- H2: 第 24 讲单神经元的 Autograd 梯度应与手工结果一致，float32 误差应很小。
- H3: 若不清空 .grad，两次相同 backward 后梯度应由 4 累加为 8。
- H4: 对于 Linear，Autograd 应满足 $dW = dY^T X, db = \sum dY, dX = dY W$ 。
- H5: 在 no_grad 或 detach 后得到的 Tensor 不应继续携带原图的梯度跟踪。

四、变量与控制

类型	内容
自变量	是否 requires_grad、是否清梯度、是否 detach/no_grad、上游梯度设置
因变量	.grad、grad_fn、梯度数值、Shape、更新前后 Loss
控制变量	固定参数、固定输入、固定随机种子、固定 dtype、相同 Loss 定义

五、实验环境

实验脚本自动输出 Python、NumPy、PyTorch 版本和 CUDA 是否可用。本次验证环境为 CPU，但 Autograd 的数学结果与是否在 CPU/GPU 上无关；浮点舍入可能有细微差异。

六、实验 1：观察计算图元数据

执行：

```
x = torch.tensor(3.0, requires_grad=True)
y = x * x
z = y + 2
```

记录：

- x.requires_grad;
- x.is_leaf;
- x.grad_fn;
- y/z.grad_fn;
- backward 前 x.grad 是否为 None。

七、实验 2：标量 Autograd 最小验证

构造：

$$L = x^2, \quad x = 3$$

解析答案：

$$\frac{dL}{dx} = 6$$

运行 loss.backward() 后检查 x.grad。

八、实验 3：复现第 24 讲单神经元梯度

使用：

$$x = [2, -1, 3], \quad w = [0.5, -2, 1], \quad b = 0.2, \quad target = 5$$

流程：

$$z = x \cdot w + b \rightarrow ReLU \rightarrow MSE$$

手工基准：

$$dw = [4.8, -2.4, 7.2], \quad db = 2.4, \quad dx = [1.2, -4.8, 2.4]$$

记录 Autograd 与手工结果的最大绝对误差。

九、实验 4：Gradient Accumulation

同一个参数 $p = 2$ ：

1. 重新 Forward $L = p^2$, backward 一次；
2. 不清梯度，再重新 Forward + backward；
3. 记录 p.grad 从 4 变为 8；
4. p.grad.zero_() 后再 backward，确认恢复为 4。

注意：普通情况下不能在同一张已释放的图上直接反复 backward。本实验每次都重新执行 Forward，模拟训练循环中的新计算图。

十、实验 5：非标量输出

构造：

$$y = [x_1^2, x_2^2, x_3^2]$$

先故意执行 y.backward()，记录报错；再使用：

$$L = \sum_i y_i$$

执行 backward。最后用 torch.autograd.grad 指定：

$$upstream = [1, 0.5, 0]$$

观察 vector-Jacobian product。

十一、实验 6：切断梯度记录

比较：

```
r = q * 3
r_detached = r.detach()
with torch.no_grad():
    s = q * 4
```

检查三者 requires_grad 与 grad_fn。

十二、实验 7: nn.Linear + MSE 梯度矩阵验证

Forward:

$$Y = XW^T + b$$

Loss: 对所有输出元素计算 mean MSE。

手工:

$$dY = \frac{2(Y - T)}{N}$$

$$dW = dY^T X, \quad db = \sum dY, \quad dX = dY W$$

Autograd:

```
loss.backward()
linear.weight.grad
linear.bias.grad
X.grad
```

比较最大误差。

十三、实验 8: backward 与参数更新是两件事

在实验 7 的梯度基础上:

```
with torch.no_grad():
    weight -= lr * weight.grad
    bias   -= lr * bias.grad
```

重新 Forward, 比较更新前后 Loss。然后调用 zero_grad(), 验证梯度已清除/置空。

十四、结果记录表

实验	预期	实际验证重点
x^2	$dL/dx = 6$	<code>x.grad</code> 是否为 6
单神经元	与第 24 讲一致	dw, db, dx 最大误差
梯度累加	$4 \rightarrow 8 \rightarrow 4$	是否需主动清梯度
非标量	直接 backward 报错	<code>sum / grad_outputs</code> 是否可用
Linear+MSE	三个矩阵公式一致	<code>Shape</code> 与数值是否一致
<code>no_grad/detach</code>	断开追踪	<code>requires_grad / grad_fn</code>

十五、结果边界与科研解释

- Autograd 对当前程序求导正确，不代表 Forward 逻辑正确；
- float32 与解析小数之间可能出现 10^{-6} 左右的舍入差异；
- 单次参数更新 Loss 下降只验证局部更新闭环，不等于模型会泛化；
- `detach/no_grad` 会改变梯度流，科研代码中必须有明确理由；
- 梯度累加可被有意用于 gradient accumulation，但必须和“忘记清梯度”的 Bug 区分。

十六、运行方式

在包含脚本的目录运行：

```
python lesson29_autograd_experiment.py
```

脚本会输出 TXT 可记录内容，并在 `figures/` 中生成三张图。

十七、本实验的科研价值

本实验建立的是以后排查训练失败所需的最低闭环：

Parameter $\rightarrow$ Forward dependency $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ `.grad` $\rightarrow$ Update

只要某个参数“不学”，就可以沿这条链逐段检查，而不是直接猜 Optimizer 或模型能力有问题。